ORCA ADE V1.0+ | MODEL CONTEXT PROTOCOL (MCP) | LINEAR & GITHUB NATIVE | ANTIGRAVITY AI AGENT

Orquestração de Agentes com Orca, Linear & MCP

Guia completo passo a passo: desde a instalação e login até a integração de múltiplos agentes de IA paralelos usando Git Worktrees isolados e ferramentas MCP nativas.

MÓDULO 01

Visão Geral da Arquitetura

Em um fluxo de engenharia de alta performance, o desenvolvedor não deve perder tempo alternando manualmente entre Jira/Linear, navegadores, terminais e branches do Git. O **Orca** atua como a central de comando unificada.

Orca (ADE)

O ambiente de desenvolvimento de agentes. Gerencia Git Worktrees isolados, terminais WebGL, preview de tela e diffs lado a lado.

Linear

A fonte da verdade para tarefas, requisitos, bugs e ciclos de entrega rápida com tracking ágil.

GitHub

Repositório de código onde branches de worktree são sincronizadas, testes rodam e Pull Requests são avaliados.

Camada MCP

Permite que o Agente de IA consulte tickets, comente no Linear e abra PRs no GitHub sem intervenção humana.

💡 Por que essa combinação é revolucionária?

Você seleciona um card no Linear, o Orca cria instantaneamente um **Git Worktree** isolado em background. O agente de IA codifica, roda os testes e abre o Pull Request no GitHub com rastreabilidade completa. Zero conflito de branches locais!

MÓDULO 02

Instalação e Login nos Serviços

1 Instalação e Primeiro Login no Orca

O Orca está disponível para Windows, macOS e Linux diretamente pelo repositório open-source `stablyai/orca`.

Comandos de Instalação por Sistema

```
# macOS (Homebrew)
brew install --cask stablyai/orca/orca

# Windows
# Baixe o instalador executável 'Orca-Setup.exe' em:
# https://github.com/stablyai/orca/releases

# Linux (Arch AUR)
yay -S stably-orca-bin
```

🔹 Ao abrir o aplicativo pela primeira vez, conclua o assistente de boas-vindas e faça login com sua conta (ou configure provedores de IA preferidos em Settings).

2 Configurar Conta e Gerar API Key no Linear

Para permitir que tanto o Orca quanto o Agente de IA interajam com suas tarefas, gere um Personal API Key.

1. Acesse linear.app e faça login no seu Workspace.
2. No canto superior esquerdo, clique no nome do seu Workspace → **Settings**.
3. No menu lateral sob **Account**, clique em **Security & Access**.
4. Role até a seção **Personal API Keys** e clique em **New API Key**.
5. Defina o rótulo como `Orca-MCP-Agent` e clique em **Create**.
6. Copie a chave (formato: `lin_api_...`) e guarde em um local seguro.

3 Gerar Personal Access Token (PAT) no GitHub

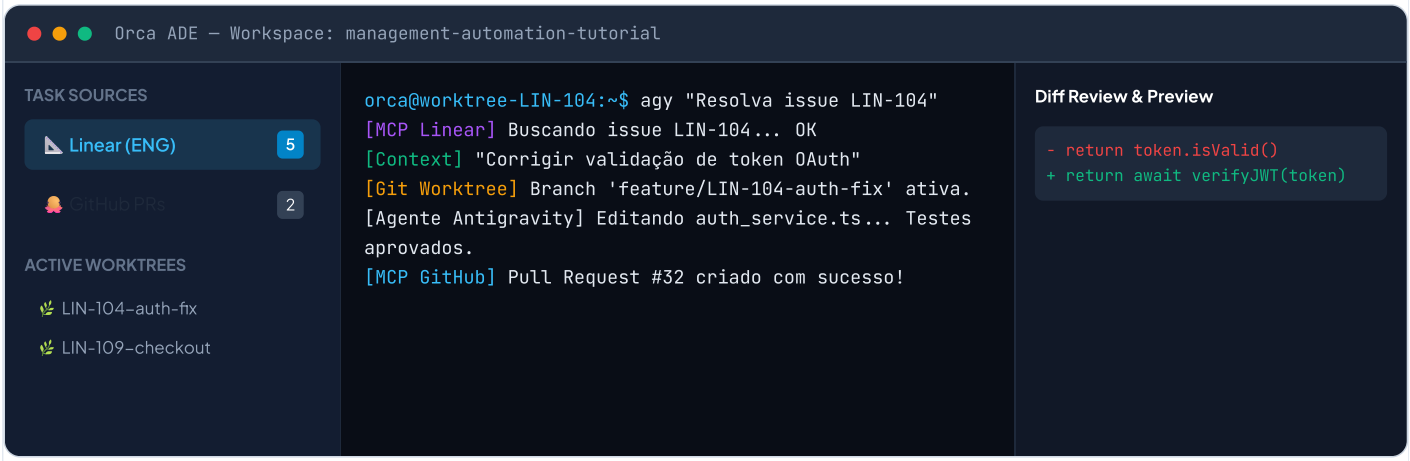
Necessário para leitura de repositórios, criação de branches e abertura de Pull Requests pelo MCP.

1. Acesse GitHub → Developer Settings → Personal access tokens.
2. Clique em **Generate new token (classic)**.
3. Dê o nome: `Orca-Agent-Integration`.
4. Marque os escopos essenciais:
   - `repo` (Acesso completo a repositórios privados e públicos)
   - `workflow` (Execução e verificação de GitHub Actions)
   - `read:org` (Listar organizações da equipe)
5. Copie o token gerado (formato: `ghp_...`).

MÓDULO 03

Conexão Nativa do Linear & GitHub no Orca

O Orca possui telas nativas dedicadas a visualizar os tickets do Linear e PRs do GitHub diretamente no cockpit da aplicação:



Configurando passo a passo no Orca

1. Abra o Orca e clique no ícone de engrenagem **Settings** (canto inferior esquerdo).
2. Acesse **Task Sources** → Marque **Linear** e **GitHub** como **ON**.
3. Acesse **Integrations**:
   - **Linear:** Clique em Conectar via OAuth ou cole sua `Personal API Key` gerada.
   - **GitHub:** Conecte sua conta do GitHub via OAuth ou insira o Personal Access Token.
4. Volte à tela inicial: os cards de tarefas aparecerão organizados em colunas (Todo, In Progress, Done).

MÓDULO 04

Configuração do MCP (Model Context Protocol)

⚠️ **Entenda a Diferença Essencial:**

A integração visual das configurações do Orca é para **você (humano)** clicar e ver cards.
A **camada MCP** é o protocolo que dá **ferramentas ao Agente de IA** para ele poder ler tarefas, pesquisar no Linear e abrir PRs no GitHub de forma 100% autônoma!

Crie um arquivo chamado `.mcp.json` na raiz do seu repositório de trabalho (ou configure via *Settings → Repository → MCP Configs* no Orca):

`.mcp.json (Configuração Pronta para Uso)`

```
{
  "mcpServers": {
    "linear": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-linear"
      ],
      "env": {
        "LINEAR_API_KEY": "lin_api_COLE_SEU_TOKEN_DO_LINEAR_AQUI"
      }
    },
    "github": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-github"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_COLE_SEU_TOKEN_DO_GITHUB_AQUI"
      }
    }
  }
}
```

Ferramenta MCP	Origem	O que a IA Consegue Fazer
`linear_search_issues`	Linear MCP	Busca issues por texto, status, tags ou projeto
`linear_get_issue`	Linear MCP	Lê detalhes, descrição completa e comentários do ticket
`linear_update_issue`	Linear MCP	Muda status para "In Progress", "In Review" ou "Done"
`create_pull_request`	GitHub MCP	Abre PR com título, descrição gerada e link para a issue
`search_code`	GitHub MCP	Procura funções, arquivos e dependências no repositório

MÓDULO 05

Conectando o Agente Antigravity no Orca

O agente **Antigravity (`agy`)** é executado diretamente nos terminais integrados do Orca, herdando as ferramentas MCP configuradas e o isolamento total da Worktree.

1 Criar a Worktree pelo Orca

No Orca, clique com o botão direito na issue desejada do Linear (ex: `LIN-204`) e selecione: **"Create Worktree from Task"**.

⚡ O Orca cria instantaneamente a pasta `.worktrees/LIN-204` e abre o terminal exatamente nessa pasta!

2 Disparar o Agente Antigravity

No terminal que acabou de abrir no Orca, execute:

Terminal Orca (PowerShell / Bash)

```
# Execução direta com prompt estruturado
agy "Consulte a issue LIN-204 no Linear via MCP. Implemente a correção solicitada, valide os testes e abra o PR no GitHub."
```

🎉 O que acontece nos bastidores:

1. O Antigravity consulta a issue no Linear via MCP e extrai os requisitos.
2. Codifica a solução nos arquivos da Worktree isolada.
3. Roda o linter e os testes locais.
4. Realiza commit e push para o GitHub.
5. Cria o Pull Request com link direto para o Linear.
6. O Orca notifica você que a tarefa está pronta para revisão visual!

MÓDULO 06

Superpoderes do Orca no Dia a Dia

Recursos essenciais que você e sua equipe devem dominar para extrair 10× mais produtividade:

Git Worktrees Paralelos

Execute 3 ou 5 agentes de IA em tarefas diferentes simultaneamente. Cada um trabalha em sua pasta sem sobrepor nem criar conflito com seus arquivos locais.

Terminais WebGL Persistentes

Terminais ultra-rápidos com scroll infinito. Se você fechar a janela ou reiniciar o PC, os processos continuam exatamente de onde pararam.

Design Mode (Inspetor Visual)

Clique em qualquer botão ou elemento da aplicação no navegador embutido do Orca. Ele copia HTML, CSS e screenshot direto para o prompt da IA.

Anotação de Diffs de IA

Revise o código gerado linha por linha antes do commit. Adicione comentários no diff e clique em "Reenviar ao Agente" para refinamento instantâneo.

MÓDULO 07

Metodologia Ágil no Linear: Uso Prático, Regras, PRD & Sprints

Para que a automação com agentes de IA (como o Antigravity) e o Orca funcione com máxima precisão, o **Linear não pode ser usado como um bloco de notas desorganizado**. Ele é o **cérebro de dados estruturados** que orienta desenvolvedores e agentes.

🎯

O Linear adota a cultura *keyboard-first* (ultra-rápido, sem burocracia de ferramentas pesadas como Jira). Nenhum código é alterado sem uma issue aberta. O ciclo de vida do card é sincronizado nativamente com o Orca e o GitHub:

Estado no Linear	Gatilho no Orca / Git	Responsável / Ação
Backlog / Triage	Ideias, débitos e bugs reportados	PM e Tech Lead priorizam e validam o DoR
Todo / Ready	Planejada no ciclo (Sprint) atual	Pronta para ser puxada por humano ou IA
In Progress	Orca cria o **Git Worktree** isolado	Agente Antigravity lê a issue via MCP e codifica
In Review	Pull Request aberto no GitHub	Agente atualiza o ticket via MCP e time faz Code Review
Done	PR mergeado na branch `main`	Fechamento automático pelo webhook do GitHub

📋

⚠️ **Regra de Ouro para Agentes de IA:**

Se uma tarefa for ambígua para uma pessoa, o Agente de IA irá **alucinar ou construir a coisa errada com altíssima velocidade**. A assertividade do código gerado pela IA é diretamente proporcional à qualidade do ticket no Linear!

❌ Títulos Proibidos (Vagos)

- `Bug no login`
- `Arrumar front-end`
- `Melhorar performance do banco`
- `Ajustes no checkout`

✅ Títulos Corretos (Taxonomia Padrão)

- `[Auth/JWT] Implementar renovação via refresh token em /auth/refresh`
- `[Checkout] Corrigir cálculo de frete para pacotes acima de 2kg`
- `[DB/Migration] Criar índice na coluna user_id em audit_logs`
- `[UI/Cart] Desabilitar botão de compra quando estoque for zero`

Template de Issue (Copie e cole no Linear)

```
### 1. Contexto & Problema
Qual o comportamento atual e por que precisa ser alterado.

### 2. Escopo Técnico & Arquitetura
- O que DEVE ser alterado (endpoints, tabelas, componentes).
- O que NÃO deve ser alterado (fronteiras de escopo para evitar refatorações indesejadas pelo agente).

### 3. Critérios de Aceitação (Definition of Done)
- [ ] O endpoint responde 200 com novo token em payload válido.
- [ ] Retorna 401 caso o token esteja revogado ou expirado.
- [ ] Testes unitários cobrem cenários de sucesso e falha.

### 4. Anexos & Referências
```

```
- Link do PRD ou documento de arquitetura no Linear Documents.
- Link do Figma / Prints / Logs de erro para reprodução.
```

📖

O PRD moderno não é um documento burocrático de 50 páginas: é o **mapa de contexto estratégico e arquitetural**. Na era da IA, ele cumpre um papel crítico:

Âncora de Contexto Macro

Uma issue isolada diz *"o que fazer"*, mas o PRD explica *"por que e como se conecta ao restante do sistema"*. Impede decisões arquiteturais isoladas que quebram o app.

Consumível via MCP

O PRD vive dentro do **Linear Documents** vinculado ao Projeto. O agente Antigravity consulta o projeto via MCP e obtém as diretrizes antes de codificar.

Delimitação In/Out Scope

Define explicitamente o que NÃO faz parte desta entrega, impedindo que o modelo de IA tente reescrever arquivos alheios ou inventar funcionalidades.

⚡

O Linear abandona a rigidez de sprints pesadas e introduz os **Cycles**: cadência contínua com foco em velocidade, previsibilidade e automação:

1 Configuração do Ciclo (Cadência de 2 Semanas)

Vá em **Team Settings → Cycles** no Linear. Ative ciclos de **2 semanas** com início às segundas-feiras. Ative o `Auto-rollover` (tarefas não finalizadas passam para o próximo ciclo com alerta) e `Auto-archive` para tarefas concluídas.

2 Fatiamento Atômico de Issues (Grooming)

Durante o refinamento, o Tech Lead fatiará o PRD em **Issues Atômicas**.
Regra de Ouro: Nenhuma issue deve demandar mais de 1 ou 2 dias de trabalho. Se a estimativa for ≥ 8 pontos, a tarefa DEVE ser dividida em sub-issues menores.

3 Planning com Velocity e Escopo Congelado

O Linear calcula automaticamente a capacidade histórica do time (*Historical Velocity*). Na reunião de Planning, selecione do Backlog apenas o montante que cabe na capacidade.
Regra do Escopo Congelado: Não se adicionam novos cards no meio do ciclo sem retirar outro de mesmo peso (exceto bugs críticos P1/Urgent).

4 Acompanhamento com Burndown Chart e Scope Graph

Acompanhe diariamente pelo painel nativo do Cycle:

- **Burndown Chart:** Mostra se o ritmo de queima de pontos está alinhado com a data final.
- **Scope Graph:** Alerta visualmente se o escopo aumentou no meio da sprint (*Scope Creep*).

MÓDULO 08

Solução de Problemas (Troubleshooting)

Sintoma	Causa Provável	Solução Rápida
Servidor MCP com status vermelho / disconnected	Node.js/npx ausente ou token inválido	Rode `node -v` no terminal. Verifique se o token no `.mcp.json` não contém espaços ou aspas extras.
"Resource not found" ao listar issues	Token sem permissão no workspace do Linear	Gere novamente o token dentro do Workspace correto em *Settings → Security & Access*.
Orca não reconhece o arquivo `.mcp.json`	Arquivo fora da raiz do projeto	Certifique-se de que `.mcp.json` está na mesma pasta raiz que você abriu no Orca.
Porta já em uso no preview local (ex: :3000)	Múltiplos worktrees rodando simultaneamente	Execute o dev server do segundo worktree em outra porta: `npm run dev -- --port 3001`.

Tutorial desenvolvido para automação ágil com Orca, Linear, GitHub e Antigravity via MCP.

Pressione `Ctrl + P` a qualquer momento para imprimir ou salvar em PDF.